

2026 - 1 지능형 로보틱스 기말 프로젝트

우주 실험실 복구 로봇 ‘ROCKY’

2271018 김은솜, 2271053 정소은



팀원 역할 분담

김은솜

- Rocky Coordinator 구현
- Nav2 자율주행 및 목표 지점 이동
- 에너지 충전 및 배양기 가동 미션 구현
- Isaac Sim 환경 구성 및 동적 장애물 구현.

정소은

- OpenCV·MediaPipe 기반 손 제스처 인식,
- ‘STOP`·`START` 명령 분류 및 ROS2 토픽 발행
- 발표 자료 및 영상 제작.

CONTENTS



프로젝트 목표 및 주제



시스템 구조



프로젝트 내용. 구현 방법



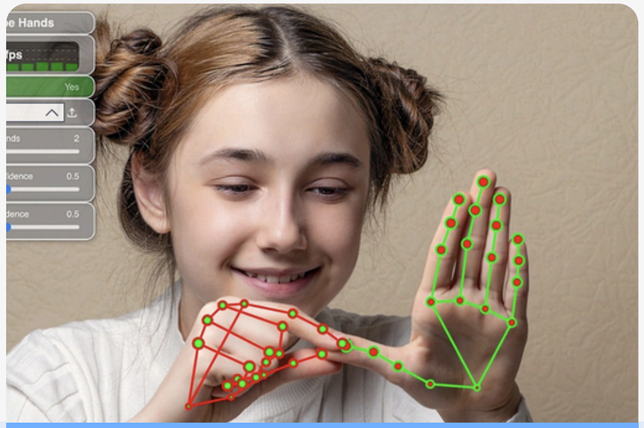
어려웠던 점 또는 개선 방향



결과 및 데모

우주 실험실 복구 로봇 ‘ROCKY’

《프로젝트 헤일메리》를 모티브로 한 우주 실험실 복구 미션
사용자의 제스처 명령을 받은 Rocky가 동적 장애물을 회피하며 에너지 스테이션을 충전하고, 배양기 장비를 가동해 실험실을 복구



제스처 기반 HRI

손 제스처를 인식하여
로봇의 출발과 정지를 제어



작업 구역 순차 자율주행

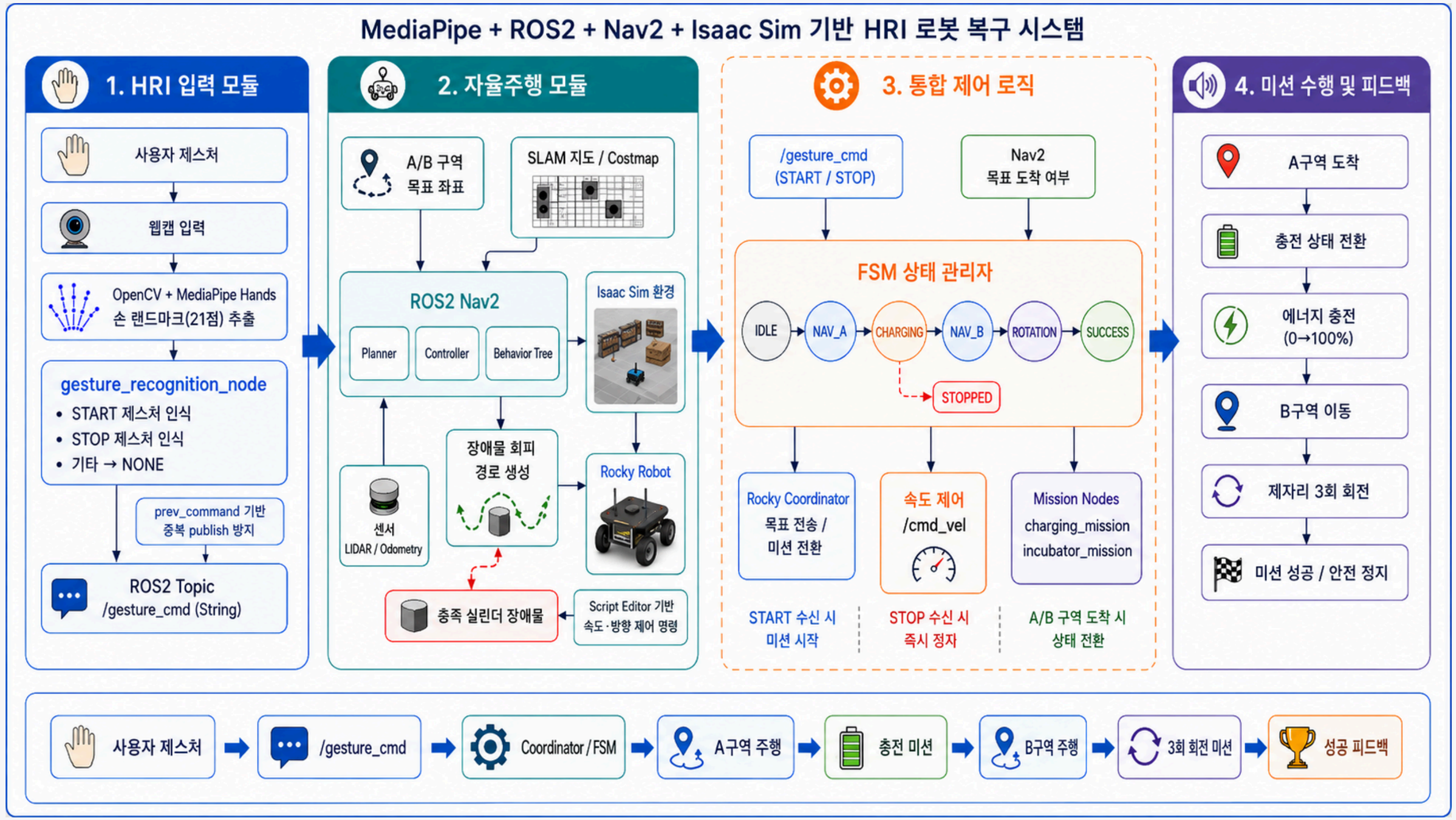
Nav2를 기반으로 동적 장애물을 회피하며
에너지 스테이션과 배양기를 순차적으로 이동



구역별 정밀 동작 수행

A구역에서 에너지를 충전하고 B구역에서
제자리 회전을 수행하여 배양기 장비 가동

시스템 아키텍처


MediaPipe + ROS2 + Nav2 + Isaac Sim 기반 HRI 로봇 복구 시스템
1. HRI 입력 모듈
사용자 제스처
웹캠 입력
OpenCV + MediaPipe Hands
손 랜드마크(21점) 추출
gesture_recognition_node
START 제스처 인식
STOP 제스처 인식
기타 → NONE
prev_command 기반
중복 publish 방지
ROS2 Topic
/gesture_cmd (String)
2. 자율주행 모듈
A/B 구역
목표 좌표
SLAM 지도 / Costmap
ROS2 Nav2
Planner
Controller
Behavior Tree
Isaac Sim 환경
센서
LIDAR / Odometry
장애물 회피
경로 생성
Rocky Robot
충족 실린더 장애물
Script Editor 기반
속도·방향 제어 명령
3. 통합 제어 로직
/gesture_cmd
(START / STOP)
Nav2
목표 도착 여부
FSM 상태 관리자
IDLE
NAV_A
CHARGING
NAV_B
ROTATION
SUCCESS
STOPPED
Rocky Coordinator
목표 전송 /
미션 전환
속도 제어
/cmd_vel
Mission Nodes
charging_mission
incubator_mission
START 수신 시
미션 시작
STOP 수신 시
즉시 정지
A/B 구역 도착 시
상태 전환
4. 미션 수행 및 피드백
A구역 도착
충전 상태 전환
에너지 충전
(0→100%)
B구역 이동
제자리 3회 회전
미션 성공 / 안전 정지
사용자 제스처
/gesture_cmd
Coordinator / FSM
A구역 주행
충전 미션
B구역 주행
3회 회전 미션
성공 피드백

제스처 기반 HRI

손 제스처를 인식하여 로봇의 출발과 정지를 제어





- **영상 데이터 입력** - 웹캠을 통한 실시간 입력을 기본으로 설계하였으며,
  시연 노트북의 웹캠 사용 제약으로 사전에 촬영한 제스처 영상을
  입력 데이터로 활용

- **손 랜드마크 추출** - OpenCV로 영상 프레임을 전처리하고
  MediaPipe Hands를 통해 21개의 손 랜드마크 검출

- **제스처 분류** - 손가락 끝점과 관절 좌표를 비교하여 손가락의 펼침 여부 판단
  - 주먹 → GO
  - 손바닥 → STOP
  - 그 외 → NONE

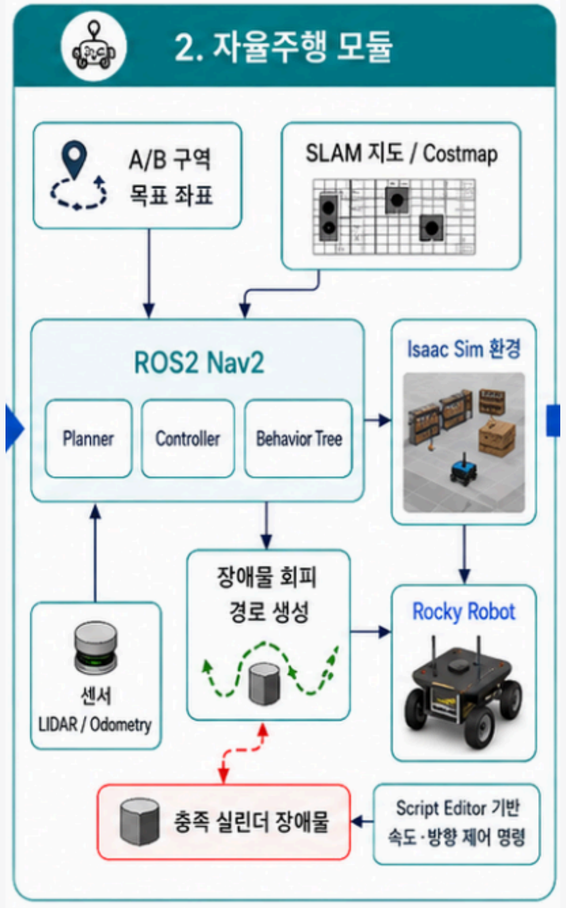
- **ROS2 명령 변환** - 인식된 제스처를 문자열 명령으로 변환하여
  /gesture_cmd 토픽으로 발행

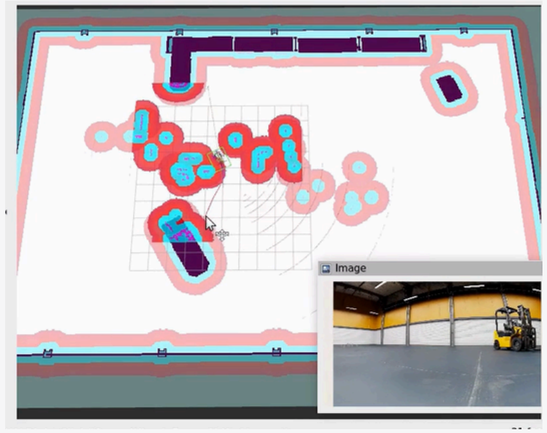
- **중복 명령 방지** - 이전 명령과 달라진 경우에만 publish하여
  불필요한 반복 전송 최소화

- **실시간 시각화** - 입력 영상에 손 랜드마크와 현재 인식 명령을 함께 표시

사용자 지정 목표점 기반 자율주행

A·B 구역의 목표 좌표를 기반으로 Rocky가 동적 장애물을 감지·회피하며
각 작업 구역까지 안전하게 자율주행

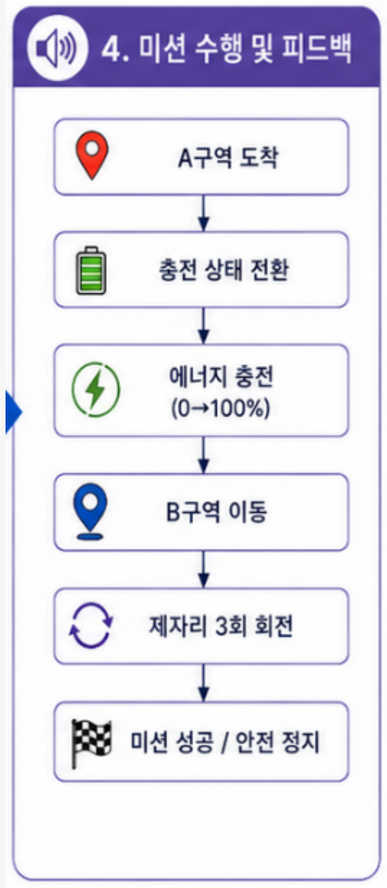




- **가상 주행 환경 구성** - Isaac Sim에 Rocky와 실험실 구조물, 동적 실린더 장애물을 배치하고 **ROS2와 연동**
- **구역별 목표 좌표 설정** - A구역과 B구역의 목표 위치를 미리 지정하여 미션 단계에 따라 순차적으로 전달
- **지도 및 위치 정보 활용** - **SLAM 지도와 Costmap, Odometry 정보**를 이용하여 로봇의 현재 위치와 주변 환경 파악
- **Nav2 기반 경로 계획** - Planner, Controller, Behavior Tree를 활용하여 **목표 구역까지의 경로** 생성 및 주행 제어
- **장애물 인식 및 회피** - LiDAR와 Costmap을 통해 정적·동적 장애물을 감지하고 충돌을 피하는 **우회 경로** 생성
- **동적 장애물 환경 구현** - Isaac Sim Script Editor를 이용해 실린더 장애물이 서로 다른 속도와 방향으로 이동하도록 구성
- **실시간 경로 재계획** - 이동 중 장애물의 위치가 변경되면 Nav2가 새로운 안전 경로를 계산하여 주행 지속

구역별 정밀 동작 수행

A구역 충전 미션과 B구역 배양기 가동 미션을 순차적으로 수행하고,
완료 후 로봇을 안전하게 정지




```
진행률 : 0%
[INFO] [1781848262.021944914] [charging_mission]: 에너지 충전
진행률 : 20%
[INFO] [1781848263.023027337] [charging_mission]: 에너지 충전
진행률 : 40%
[INFO] [1781848264.024025851] [charging_mission]: 에너지 충전
진행률 : 60%
[INFO] [1781848265.025017503] [charging_mission]: 에너지 충전
진행률 : 80%
[INFO] [1781848266.025939084] [charging_mission]: 에너지 충전
진행률 : 100%
[INFO] [1781848267.026944737] [charging_mission]: 에너지 100%
충전 완료
```

```
som@glab:~$ source ~/IsaacSim-ros_workspaces/humble_ws/install/
setup.bash && ros2 run rocky_mission_controller incubator_missi
on
[INFO] [1781848175.676190008] [incubator_mission]: Incubator Mi
ssion 노드 대기 중
[INFO] [1781848288.549961790] [incubator_mission]: 배양기 가동.
제자리 회전을 수행합니다.
```

- **A구역 도착** - **Nav2 자율주행**을 통해 Rocky가 **에너지 스테이션**으로 이동하고 목표 지점 도착 여부 확인
- **충전 상태 전환** - A구역 도착 신호를 수신하면 주행 상태에서 **충전 미션 상태**로 전환
- **에너지 충전** - 타이머를 이용해 **충전 진행률을 20%씩 증가**시키며 0%부터 100%까지 출력
- **B구역 이동** - 충전이 완료되면 Rocky Coordinator가 **B구역의 목표 좌표를 Nav2에 전달하여 이동** 시작
- **배양기 가동** - B구역 도착 후 /cmd_vel을 제어하여 **Rocky가 제자리에서 3회 회전**
- **미션 완료 및 정지** - 회전 미션이 끝나면 **성공 상태로 전환**하고 로봇의 속도를 0으로 설정하여 안전하게 **정지**

어려웠던 점

- 초기에는 ROS2 토픽과 Isaac Sim Action Graph를 연동하여 동적 장애물을 제어하고자 함
- Action Graph 노드 구성과 ROS2 Bridge 설정 과정이 복잡하여 구현에 어려움 발생
- 장애물의 움직임을 안정적으로 제어하고 ROS2와 정상적으로 연동하는 데 많은 시간이 소요됨
- 프로젝트 일정과 구현 안정성을 고려하여 Script Editor 기반 방식으로 변경
- Script Editor에서 각 실린더의 속도, 이동 방향, 이동 거리를 직접 설정하여 동적 장애물 구현

개선할 점

- Isaac Sim Action Graph와 ROS2 Bridge를 활용한 장애물 제어 기능 구현
- 동적 장애물을 ROS2 토픽으로 제어하여 로봇과 장애물이 동일한 ROS2 환경에서 상호작용하도록 개선
- 현재의 고정된 A·B 구역 순차 주행을 다중 Waypoint 기반 주행으로 확장
- 상황에 따라 이동 경로를 변경하거나 작업 우선순위를 결정하는 기능 추가
- 실제 서비스 로봇과 유사한 유연한 미션 수행 시스템으로 발전

발표 및 데모 영상 링크

HTTPS://YOUTU.BE/NVYEOLPVWDW

참고 자료

- [NVIDIA Isaac Sim Quick Install] (https://docs.isaacsim.omniverse.nvidia.com/5.1.0/installation/quick-install.html)
- [MediaPipe Hands] (https://ai.google.dev/edge/mediapipe/solutions/vision/hand_landmarker)
- [ROS2 Humble] (https://docs.ros.org/en/humble/index.html) * [Nav2 Documentation](https://docs.nav2.org/)
- [NVIDIA ROS 2 Tutorials] (https://docs.isaacsim.omniverse.nvidia.com/latest/ros2_tutorials/tutorial_ros2_navigation.html)